FACULTY OF ENGINEERING AND TECHNOLOGY

CEF 440: INTERNET PROGRAMMING AND MOBILE PROGRAMMING

Group members

Name	Matricule
Suh Brandon Njuiebu	FE23A143
Akumo Lucien Ndifor	FE23A010
Formin Rollins Nkengfua	FE23A058
Rosy Winko Nchako	FE23A138
Munongo Bryce Munongo Mukete Jnr	FE23A099

Course Instructor: Dr Nkemeni Valery

Task 1

Q1: Major Types of Mobile Apps: Native, PWA, and Hybrid

Introduction:

Comparing and understanding Native, Hybrid, and PWA technologies is essential for balancing performance, development costs, and speed-to-market. This analysis ensures the chosen architecture aligns with specific project goals, hardware requirements, and the desired user experience across various devices.

1. Native Applications

Native apps are developed specifically for a single mobile operating system (OS). Because they are built using the OS's "mother tongue," they have a direct line of communication with the phone's hardware.

Development Languages:

- iOS: Swift or Objective-C.
- Android: Kotlin or Java.

Performance: This is the "Gold Standard." Since the code is compiled specifically for the device's processor, animations are fluid (60 frames per second), and complex tasks like video editing or AR filters run smoothly.

Hardware Access: Full and immediate access to the GPS, Camera, Accelerometer, Bluetooth, and Contact list.

Distribution: Users must download these from the Apple App Store or Google Play Store.

Example: Spotify or Pokémon GO. These apps require high performance and background processing that only native code can reliably provide.

2. Progressive Web Apps (PWAs)

A PWA is essentially a website that has been "supercharged" to behave like an app. You don't download it from a store; instead, you visit a URL in your browser and "Add to Home Screen."

- **Technology Stack:** HTML5, CSS, and JavaScript.
- **Service Workers:** These are scripts that run in the background. They allow the app to work offline by caching (saving) data and sending push notifications even when the browser is closed.
- **The "Manifest" File:** A simple JSON file that tells the phone, "Treat this website like an app." It defines the icon, the splash screen, and hides the browser's URL bar.
- **Pros:** No 30% commission to Apple/Google; instant updates (no waiting for app store approval); very low storage space required.
- **Cons:** Cannot access advanced hardware like FaceID, low-level Bluetooth, or the proximity sensor.

Example: Starbucks or Forbes. Their PWAs allow users to browse menus or read articles even with a poor internet connection without needing a 100MB download.

3. Hybrid Applications

Hybrid apps are a middle ground. They are essentially web apps (HTML/JS) "wrapped" inside a native container. Imagine a native app that is just one big window (a WebView) displaying a website.

- **Technology Stack:** Usually built with frameworks like Ionic or Apache Cordova.
- **How it works:** The code is written once. When the app runs, the native wrapper provides an "interface" (a bridge) that allows the web code to talk to the phone's camera or file system.
- **Development Speed:** Very fast. One team of web developers can build one app that runs on both iPhone and Android simultaneously.
- **Performance:** This is the weakest point. Because there is a "bridge" between the web code and the hardware, there is often a slight delay or "jank" in animations.

Example: Amazon or Evernote. These apps have a lot of text and lists where absolute top-tier gaming performance isn't necessary, but being on every platform is vital.

Plain comparison table on different PL features

Features	Native	PWA	Hybrid
Performance	High	Low/medium	Medium
Dev cost	High	Low	Medium
Appstore presence	Yes	No	Yes
Hardware Access	Full	Limited	Good

2. Review and Comparism of Different Mobile App Programming Languages

1. **Native Platform:** Native language offers the best performance and intermediate access to new OS features. Examples:

- **Java:** Java was the first programming language for Android app development which is now considered a Legacy Language mobile.
- **Kotlin:** Kotlin is the current programming language for all Android apps which is now referred to as King of Android, it surpassed Java as the preferred language because of how secure it is.

- **Swift:** This is the standard programming language for IOS.

2. **Cross-Platform (One Codebase):** Cross-platforms allows us to write one set of code that runs for both IOS and Android. Examples:

- **Dart:** Dart is a programming language used for writing codes on the flutter framework, it is the most popular cross-platform used for both IOS and Android development.
- **Javascript/ Typescript:** This is a simple programming language where Js provides core logic and flexibility to make apps work while TS adds a layer of strict rules to ensure that logic is reliable and professional.

3. **Hybrid (Web-to-Native) Platform:** Hybrid is essentially a website disguised as a mobile app. It can develop apps for both Android and IOS. Examples of hybrid languages:

- **HTML:** Used to build structure and content
- **CSS:** Used to design the platform
- **Javascript/ Typescript:** Used to handle logic Examples of hybrid platforms:

- **Ionic:** This is the most popular platform for building hybrid. It provides a library of mobile components that look like real Android and IOS parts.
- **Capacitor:** This is the engine that lets JS interact with the phone hardware.
- **Apache Cordova:** It is used to wrap web code into a mobile package.

Comparison between Native Platforms

Feature	Java	Kotlin	Swift
Platform	Android (legacy)	Android (modern)	IOS
Performance	Used to maintain old apps	Used for modern apps	Used for both old and modern apple apps
Code Length	Very long	Short and Concise	Short and Concise
Security	Low (cracks easily)	High	High
Learning Curve	Hard/ Moderate	Moderate	Easy/ Moderate

Comparison between Cross-Platforms

Features	Flutter	React Native
Language	Dart	JS/ TS

Philosophy	It draws every pixel itself	JS provides the logic to make the app work while TS provides the strict rule for the logic
Risk	Google might kill it	Meta's messy updates
Cost	Expensive	Cheaper

Comparison between Hybrid Platforms

Features	Ionic	Capacitor	Apache Cordova
Language	TS/ JS	TS /JS	JS
Role	The UI library	The native engine	The legacy wrapper
Hardware access	Uses capacitor/ cordova	Easy and direct	Complex

Q3. Mobile App Development Frameworks: Review and Comparison

1. Introduction

Mobile app development frameworks are tools that help developers build mobile applications more easily and quickly. Instead of writing everything from scratch, developers can reuse code, which saves time and reduces cost. These frameworks are especially important today because apps are expected to run on both Android and iOS platforms.

This report reviews and compares popular mobile app development frameworks based on key features such as programming language, performance, cost and time to market, user interface (UI) and user experience (UX), complexity, community support, and where they are commonly used.

2. Overview of Mobile App Frameworks

Mobile frameworks can generally be grouped into:

- **Native frameworks** (built specifically for one platform)
- **Cross-platform frameworks** (one codebase for multiple platforms)
- **Hybrid frameworks** (web-based apps wrapped in a mobile container)

This report focuses on widely used cross-platform frameworks because they help reduce development time and cost.

3. Comparison of Popular Frameworks

3.1 Flutter

Flutter is a framework developed by Google that uses the Dart programming language.

- **Language:** Dart
- **Performance:** Very high because it compiles directly into native code, making it faster than many other frameworks
- **Cost & Time to Market:** Low cost and fast development since one codebase works for both Android and iOS
- **UX/UI:** Excellent — provides customizable widgets and smooth designs
- **Complexity:** Moderate, especially for beginners who are new to Dart
- **Community Support:** Growing very quickly

Where it is used:
Flutter is suitable for high-performance apps and apps that need attractive designs. For example, it has been used in apps like Google Ads.

3.2 React Native

React Native is developed by Meta and allows developers to use JavaScript.

- **Language:** JavaScript
- **Performance:** High, but slightly lower than Flutter because it uses a bridge to communicate with native components
- **Cost & Time to Market:** Low cost and fast, especially for developers already familiar with JavaScript
- **UX/UI:** Good, since it uses native components
- **Complexity:** Easy to learn compared to other frameworks
- **Community Support:** Very strong and widely used

Where it is used:
React Native is commonly used for social media and business apps. It is used in apps like Facebook and Instagram.

3.3 Xamarin

Xamarin is a Microsoft framework that uses C# and the .NET platform.

- **Language:** C#
- **Performance:** Near-native performance
- **Cost & Time to Market:** Moderate, as it may take longer to learn and develop compared to JavaScript-based frameworks
- **UX/UI:** Good, but less flexible than Flutter
- **Complexity:** Moderate to high
- **Community Support:** Strong due to Microsoft support

Where it is used:
Xamarin is mainly used for enterprise-level applications, especially in companies that already use Microsoft technologies.

3.4 Ionic

Ionic is a hybrid framework that builds apps using web technologies.

- **Language:** HTML, CSS, JavaScript
- **Performance:** Medium to low because it runs inside a web view
- **Cost & Time to Market:** Very low cost and very fast to develop
- **UX/UI:** Decent, but may not feel fully native
- **Complexity:** Easy to learn
- **Community Support:** Strong

Where it is used:
Ionic is best for simple apps, prototypes, or applications where performance is not the main concern.

4. Comparative Analysis

Feature	Flutter	React Native	Xamarin	Ionic
Language	Dart	JavaScript	C#	HTML/CSS/JS
Performance	Very High	High	High	Medium
Cost & Time	Low & Fast	Low & Fast	Moderate	Very Low
UX/UI	Excellent	Good	Good	Average
Complexity	Moderate	Easy	Moderate	Easy
Community	Growing	Very Strong	Strong	Strong

Key Observations

- Flutter performs better than React Native because it compiles directly to native code, while React Native depends on a bridge.
- React Native is easier to learn due to JavaScript being widely known.

- Xamarin is more suitable for enterprise environments but is slightly more complex.
- Ionic is the fastest to develop with, but performance is lower compared to others.

5. Discussion

From the comparison, it is clear that cross-platform frameworks help reduce development time because developers only need to write one codebase for multiple platforms. This significantly improves time to market and reduces cost.

However, the choice of framework depends on the type of application:

- For high performance and design → Flutter is a strong choice
- For ease of development → React Native is more suitable
- For enterprise systems → Xamarin works well
- For simple or low-budget apps → Ionic is ideal

6. Conclusion

In conclusion, mobile app development frameworks play an important role in modern application development by reducing cost, saving time, and improving efficiency. Among the frameworks discussed, Flutter and React Native stand out due to their balance between performance, development speed, and strong community support. However, the best framework depends on the specific needs of the project, including complexity, budget, and performance requirements.

4. Mobile Application Architectures and Design Patterns.

Mobile application architecture is the overall structure of a mobile application, including how the user interface, business logic, data, and services interact. A good architecture makes an app easier to maintain, test, scale, and update. Design patterns are reusable solutions to common software design problems.

Main Layers of a Mobile Application

Most mobile applications are divided into three main layers:

- Presentation Layer: Contains the user interface, screens, buttons, and user interactions.
- Business Logic Layer: Handles calculations, validation, and application rules.
- Data Layer: Manages local databases, APIs, cloud services, and storage.

Types of Mobile Application Architectures

a) Monolithic Architecture

Here, all parts of the mobile application are developed as a single unit.

Advantage: Simple to build for small apps, easy deployment.

Disadvantage: Difficult to maintain and scale.

b) Layered Architecture

The application is separated into layers such as presentation, business logic and data.

Advantage: Better organization, easier testing and maintenance.

Disadvantage: More complex than monolithic architecture.

c) Client–Server Architecture

The mobile app acts as the client and communicates with a backend server through APIs.

Advantage: Centralized data, supports multiple users.

Disadvantage: Requires internet connection and secure communication.

d) Microservices Architecture

The backend is divided into small independent services.

Advantage: High scalability, easy to update individual services.
Disadvantage: More difficult to design and manage.

Common Mobile Design Patterns

a) MVC (Model–View–Controller)

Model:
Handles data and business logic.

View: Displays information to the user.

Controller: Connects the model and the view.

Advantages: Easy to understand and widely used.

Disadvantages: Controller can become too large in complex apps.

b) MVP (Model–View–Presenter)

Presenter: acts between the model and the view.

Advantages: Easier testing and separation of concerns.

Disadvantages: More classes and files are required.

c) MVVM (Model–View–View Model)

The View Model: stores presentation logic and communicates with the view through data binding.

Advantages: Clean separation, good for modern Android and iOS frameworks.
Disadvantages: Slightly more difficult to learn.

d) VIPER

VIPER: stands for View, Interactor, Presenter, Entity and Router.

Advantages: Highly organized and testable.
Disadvantages: Too complex for small applications.

e) Repository Pattern

Provides a single place to access data from databases, APIs, or local storage.

Advantages: Easier data management and testing.

f) Singleton Pattern

Ensures that only one instance of an object exists.

Example: A database manager or network manager.

Advantages: Saves memory and provides global access.

Comparison of Major Design Patterns

- **MVC** is suitable for simple applications but becomes difficult to manage in large projects.
- **MVP** provides better separation than MVC and is easier to test.
- **MVVM** is one of the most popular patterns for modern mobile applications because it works well with data binding and reactive programming.
- **VIPER** offers the highest level of organization but is mainly used in large iOS projects because of its complexity.

Choosing the Right Architecture

The choice of architecture depends on the size and purpose of the application:

- Small apps: **MVC** or simple layered architecture
- Medium apps: **MVP** or **MVVM**
- Large enterprise apps: **MVVM** with repository pattern or microservices backend
- Very complex iOS apps: **VIPER**

Conclusion

Mobile application architecture and design patterns are essential for building high-quality mobile apps. A well-designed architecture improves maintainability, performance, and scalability. Among the common patterns, MVVM is widely preferred in modern development, while MVC and MVP are suitable for smaller applications. Developers should choose the architecture that best matches the size, complexity, and future requirements of their desired mobile application.

Q5. Study on Collecting and Analysing User Requirements for a Mobile Application (Requirements Engineering)

1. Introduction

Requirement engineering is a crucial phase in mobile application development. It focuses on identifying, collecting, and analyzing user needs to ensure that the final application meets user expectations and solves real-world problems effectively.

2. Collecting User Requirements

User requirements can be gathered using several techniques:

- Interviews: Direct communication with users to understand their needs and expectations.
- Surveys and Questionnaires: Useful for collecting data from a large number of users quickly.
- Observation: Studying how users interact with existing systems to identify pain points.
- Focus Groups: Group discussions that provide diverse user opinions and ideas.
- Brainstorming: Encourages creative thinking to generate new features and solutions.

3. Analysing User Requirements

After collection, requirements must be carefully analyzed:
Classification:

- Functional Requirements – Describe what the system should do.
- Non-functional Requirements Describe system qualities such as performance, security, and usability.
- Prioritization: Requirements are ranked based on importance and feasibility.

Modeling Tools:

- Use case diagrams
- User stories

These tools help in clearly representing requirements for developers and stakeholders.

4. Validation of Requirements

Validation ensures that the collected requirements are accurate and complete

Examples:

- Prototyping: Creating a basic version of the app to gather feedback.
- User Feedback Sessions: Users review and confirm requirements.

- Reviews and Testing: Checking for consistency and completeness.

5. Conclusion

Effective requirements engineering ensures that mobile applications are user-centered and meet desired objectives. Proper collection and analysis of requirements reduce development risks, improve software quality, and increase user satisfaction.

6. Estimating Mobile App Development Cost

Introduction

Estimating the cost of a mobile application is one of the most critical steps in the development lifecycle. It involves more than just guessing a price; it requires a detailed analysis of the project's scope, the technical complexity, and the resources needed. A correct estimate helps stakeholders manage budgets, set realistic launch dates, and decide which features are essential for a Minimum Viable Product (MVP) versus what can be added later.

The Core Estimation Formula

In the professional world, most development costs are calculated using a simple but effective equation:

Cost = (Total development hour) X (hourly rate of developers) + External Cost

- Total Development Hours: The sum of time spent on design, coding, testing, and project management.

- Hourly Rate: This varies wildly based on where the developers are located and their level of expertise.

- External Costs: These include third-party services like server hosting (AWS/Google Cloud), API subscriptions (Google Maps API), and app store registration fees.

Key Factors Influencing Cost

1. App Complexity

Complexity is determined by the number of features and how "smart" the app needs to be.

Simple Apps (MVP): Basic functionality, 5–8 screens, and simple user login.

Example: A Simple Flashlight App or a Basic To-Do List.

Est. Time: 300–600 hours.

Medium Complexity: Includes custom UI, integration with APIs, payment gateways, and real-time notifications.

Example: A Food Delivery App (like UberEats) or a Fitness Tracker.

Est. Time: 600–1,200 hours.

Complex Apps: Features like AI/Machine Learning, heavy data processing, multi-language support, and high-security encryption.

Example: A Banking App or a Social Media Platform (like Instagram/TikTok).

Est. Time: 1,200+ hours.

2. Choice of Platform

Native (iOS and Android): Most expensive because you essentially build two separate apps.

Cross-Platform (Flutter/React Native): More cost-effective as you use one codebase to target both platforms.

PWA: Cheapest option as it runs in the web browser and doesn't require app store approval.

3. Development Team Location (2026 Benchmarks)

The "Hourly Rate" part of the formula depends on where the talent is based.

North America (USA/Canada): $80 – $150+ / hr.

Western Europe (UK/Germany): $60 – $110 / hr.

Eastern Europe (Poland/Ukraine): $35 – $70 / hr.

Africa/Asia (India/Cameroon): $15 – $45 / hr.

Cost Breakdown by Development Phase

A project budget is usually divided across the following stages:

Percentage of Budget	Activities
10%	Market research, requirements gathering and creating user stories.
15-20%	Creating wireframes, prototypes and final visual brand.
50-60%	The actual coding of the front-end, backend and API integrations.
15%	Testing for bugs, security vulnerabilities, and performance issues.
5%	Deployment or publishing to PlayStore of Appstore.

Hidden and Ongoing Costs

The cost does not end when the app is launched. You must budget for:

1. App Maintenance: Roughly 15%–25% of the original cost per year. This covers bug fixes, OS updates (e.g., updating for Android 15), and server costs.

2. Marketing: If people don't know the app exists, it won't be used. This can often cost as much as the development itself.

3. Third-Party Fees: Apple Developer Program: $99/year.

Google Play Console: $25 which is a one-time fee.

4. SMS Services: Using Twilio for phone verification costs per message sent.